ШКОЛА АНАЛИТИКОВ HH.RU · ЛЕКЦИЯ 2

Продвинутый SQL

Хэндбук: справочник конструкций с готовыми примерами

PostgreSQL · блок «Инструменты анализа и визуализации данных»

СОВЕТ

Это **справочник, а не конспект**: по каждой теме — короткое определение, таблица и пример кода, который можно скопировать и подставить свои таблицы.

Подробные объяснения — в конспекте лекции. Если нужно быстро вспомнить синтаксис — сразу листайте на последнюю страницу, там шпаргалка.

Содержание

1. Как строить сложный запрос: подзапросы, CTE, UNION ALL, гранулярность
2. Оконные функции: синтаксис, рамки, где их можно использовать
3. Классы оконных функций: агрегаты, ранжирование, LAG/LEAD, FIRST/LAST_VALUE
4. Строки, ARRAY и JSONB
5. Полезные PostgreSQL-приёмы: EXISTS, DISTINCT ON, generate_series
6. Производительность: базовые привычки
7. Шпаргалка

1. Как строить сложный запрос

Сложный SQL — это последовательность понятных шагов. Перед кодом определите шаги словами и для каждого ответьте: **что означает одна строка** после него.

Какой инструмент для какой задачи

Задача	Инструмент
Разбить расчёт на этапы, один короткий шаг	`подзапрос в FROM`
Несколько этапов, результат нужен повторно, важна читаемость	`CTE (WITH)`
Сложить однотипные события из разных таблиц	`UNION ALL`
Сложить и убрать полностью совпадающие строки	`UNION`
Добавить к строке колонки из другой таблицы	`JOIN`

Подзапрос: где и что возвращает

Где	Возвращает	Пример
`SELECT`	одно значение на строку	`(SELECT COUNT(*) FROM calls c WHERE c.response_id = r.response_id)`
`FROM`	**таблицу — отдельный этап**	**`FROM (SELECT ... GROUP BY response_id) AS message_stats`**
`WHERE`	список для проверки	`WHERE response_id IN (SELECT response_id FROM calls)`

Шаблон: каркас из CTE

```
WITH messages_7d AS (       -- 1 строка = сообщение
    SELECT ...
),
calls_7d AS (               -- 1 строка = звонок
    SELECT ...
),
events AS (                 -- 1 строка = коммуникация
    SELECT ... FROM messages_7d
    UNION ALL
    SELECT ... FROM calls_7d
),
timeline AS (               -- то же + порядок событий
    SELECT ... FROM events
)
SELECT ...                  -- 1 строка = отклик / работодатель
FROM timeline;
```

Шаблон: события из двух таблиц через UNION ALL

```sql
SELECT response_id,
       message_id::text AS event_id,   -- типы колонок должны совпадать
       sent_at          AS event_at,
       'message'        AS event_type
FROM messages
WHERE sender_role = 'employer'
UNION ALL
SELECT response_id,
       call_id::text,
       call_at,
       'call'
FROM calls
WHERE caller_role = 'employer';
```

ПОДВОДНЫЙ КАМЕНЬ

Перед каждым JOIN спросите: **что означает одна строка слева и справа, и что будет после соединения?**

3 сообщения × 2 звонка по одному отклику дают 6 строк — и SUM по звонкам завышается втрое. Сначала приведите каждый источник к нужной гранулярности, потом соединяйте.

СОВЕТ

Читаемый SQL: имена объясняют содержимое (`messages_7d`, не `t1`), команды капсом, одинаковые отступы, комментарии объясняют «почему». CTE — столько, сколько шагов в логике, не больше.

2. Оконные функции

ОПРЕДЕЛЕНИЕ

Оконная функция считает по группе строк, но **сохраняет каждую исходную строку**. GROUP BY: 4 звонка → 2 строки. Окно: 4 звонка → 4 строки, а среднее по работодателю — рядом.

Синтаксис

```
<function> OVER (
    PARTITION BY ...                    -- внутри какой группы
    ORDER BY ...                        -- в каком порядке
    ROWS / RANGE BETWEEN ... AND ...    -- какой фрагмент
)
```

Часть	Если не указать
PARTITION BY	вся выборка — одна партиция
ORDER BY	порядка нет, окно = вся партиция
рамка (при наличии ORDER BY)	RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

Границы рамки

Граница	Значение
UNBOUNDED PRECEDING	начало партиции
N PRECEDING	N строк (ROWS) или N единиц значения (RANGE) назад
CURRENT ROW	текущая строка (для RANGE — вместе со строками с тем же значением ORDER BY)
N FOLLOWING	N вперёд
UNBOUNDED FOLLOWING	конец партиции

ROWS или RANGE

	ROWS	RANGE
Считает	физические строки	значения ORDER BY
Пример	текущая и две предыдущие строки	все звонки за 7 дней до текущего
Одинаковые значения ORDER BY	останавливается на текущей строке	захватывает всех «соседей» с тем же значением

```
-- скользящее среднее по 3 последним звонкам
AVG(call_duration) OVER (PARTITION BY employer_id ORDER BY call_at
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)

-- среднее за 7 дней до текущего звонка
AVG(call_duration) OVER (PARTITION BY employer_id ORDER BY call_at
    RANGE BETWEEN INTERVAL '7 days' PRECEDING AND CURRENT ROW)
```

СОВЕТ

Два события в одно время? Добавьте tie-breaker: `ORDER BY event_at, event_type, event_id`.

Результат зависит от рамки — **задавайте её явно**.

Где можно использовать

Окна доступны в **SELECT и ORDER BY**. В WHERE — нельзя: WHERE выполняется раньше окон.

```
FROM → WHERE → GROUP BY → HAVING → окна → ORDER BY
```

```
-- фильтр по результату окна: через CTE
WITH ranked AS (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY response_id
                              ORDER BY event_at, event_type, event_id) AS rn
    FROM events
)
SELECT response_id, event_id
FROM ranked
WHERE rn = 1;
```

ПОДВОДНЫЙ КАМЕНЬ

`QUALIFY` фильтрует после окон без CTE, но **в PostgreSQL его нет** (есть в Snowflake, BigQuery, Databricks, DuckDB).

3. Классы оконных функций

Обзор

Функция	Что возвращает	Типичная задача
`SUM / AVG / COUNT / MIN / MAX + OVER()`	агрегат по партиции или накопительно	всего событий в отклике; событий к текущему моменту
`ROW_NUMBER()`	уникальный номер 1, 2, 3, 4	первое событие отклика
`RANK()`	1, 2, 2, 4 — с пропуском	рейтинг «сколько мест выше»
`DENSE_RANK()`	1, 2, 2, 3 — без пропуска	рейтинг «уровень значения»
`NTILE(n)`	номер группы из n	квартили, сегменты
`LAG / LEAD`	значение из предыдущей / следующей строки	время с прошлого события, следующий канал
`FIRST_VALUE / LAST_VALUE`	первое / последнее значение во фрейме	первый и последний канал связи

Агрегат: всего и накопительно

```
COUNT(*) OVER (PARTITION BY response_id) AS total_events,        -- 4, 4, 4, 4
COUNT(*) OVER (PARTITION BY response_id
               ORDER BY event_at, event_type, event_id
               ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS events_so_far                                               -- 1, 2, 3, 4
```

Ранжирование

processed_share	ROW_NUMBER	RANK	DENSE_RANK	NTILE(2)
80	1	1	1	1
70	**2**	**2**	**2**	**1**
70	**3**	**2**	**2**	**2**
60	4	4	3	2

```
DENSE_RANK() OVER (
    PARTITION BY professional_role
    ORDER BY processed_share DESC, median_first_contact_hours
) AS place

NTILE(4) OVER (ORDER BY median_first_contact_hours) AS speed_quartile  -- 1 = быстрые
```

СОВЕТ

Всегда задавайте ORDER BY. Выбор RANK или DENSE_RANK — **часть бизнес-правила рейтинга**. NTILE: если строки не делятся нацело, лишние уходят в первые группы (10 на 4 → 3, 3, 2, 2).

LAG и LEAD

```
LAG(column, offset, default)    -- offset по умолчанию 1, default — NULL
LEAD(column, offset, default)

event_at - LAG(event_at) OVER (
    PARTITION BY response_id
    ORDER BY event_at, event_type, event_id
) AS time_from_previous           -- interval; у первой строки NULL
```

Время в часах: `EXTRACT(EPOCH FROM <interval>) / 3600`. LAG и LEAD **не зависят от рамки** ROWS/RANGE.

FIRST_VALUE и LAST_VALUE

```
FIRST_VALUE(event_type) OVER (
    PARTITION BY response_id ORDER BY event_at, event_type, event_id
) AS first_event_type,
LAST_VALUE(event_type) OVER (
    PARTITION BY response_id ORDER BY event_at, event_type, event_id
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING   -- обязательно!
) AS last_event_type
```

ПОДВОДНЫЙ КАМЕНЬ

Без расширенной рамки LAST_VALUE вернёт **тип текущего события**: рамка по умолчанию заканчивается на текущей строке.

4. Строки, ARRAY и JSONB

Думайте о преобразованиях: **строки → строка, строка → массив, массив → строки, JSONB → отдельные поля**.

Строки

Конструкция	Пример	Результат
`SPLIT_PART(str, sep, n)`	`SPLIT_PART('employer|101|mobile_app', '|', 2)`	'101' (текст)
`STRING_AGG(col, sep ORDER BY …)`	`STRING_AGG(event_type, ' → ' ORDER BY event_at)`	message → call → call
`REGEXP_REPLACE(str, re, new, 'g')`	`REGEXP_REPLACE('+7 (960) 123-45-67', '[^0-9]', '', 'g')`	79601234567
`~` / `~*`	`professional_role ~* 'аналитик'`	совпадение (~* — без учёта регистра)
`!~` / `!~*`	`status !~ '^test'`	не совпадает

СОВЕТ

SPLIT_PART всегда возвращает текст — приводите тип: `::integer`. Хватает LIKE / ILIKE или разделителя — не пишите регулярку.

ARRAY

Конструкция	Что делает	Пример → результат
`ARRAY['a', 'b']`	создать массив	`ARRAY['message', 'call'] → {message,call}`
`ARRAY_AGG(col ORDER BY …)`	собрать значения группы в массив	`→ {message,message,call,call}`
`CARDINALITY(arr)`	число элементов	`→ 4`
`value = ANY(arr)`	есть ли значение	`'call' = ANY(event_sequence) → true`
`STRING_TO_ARRAY(str, sep)`	строка → массив	`STRING_TO_ARRAY('a,b', ',') → {a,b}`
`ARRAY_TO_STRING(arr, sep)`	массив → строка	`→ 'message → call'`
`UNNEST(arr) WITH ORDINALITY`	массив → строки + позиция	`1 строка → 4 строки`

```
SELECT rs.response_id, t.event_type, t.position
FROM response_sequences AS rs
CROSS JOIN UNNEST(rs.event_sequence)
    WITH ORDINALITY AS t(event_type, position);
```

ПОДВОДНЫЙ КАМЕНЬ

UNNEST меняет гранулярность. После разворачивания снова проверьте, что означает одна строка, прежде чем делать JOIN или агрегацию.

JSONB

Оператор	Что делает	Пример
`->`	значение как JSONB	`message_meta -> 'device'`
`->>`	значение как текст	`message_meta ->> 'channel'`
`(... ->> ...)::type`	текст → нужный тип	`(message_meta ->> 'is_read')::boolean`
`?`	есть ли ключ	`message_meta ? 'is_read'`
`@>`	содержит ли структуру	`message_meta @> '{"channel": "mobile_app"}'`
`-> ... ->>`	вложенное поле	`message_meta -> 'device' ->> 'os'`

СОВЕТ

Нет ключа → `->>` вернёт NULL. Отличить «ключа нет» от «значение NULL» помогает оператор `?`.

Выбирайте **JSONB**, если не нужно хранить JSON ровно в исходном виде. ARRAY — для однотипных значений, JSONB — для объекта с разными параметрами.

5. Полезные PostgreSQL-приёмы

EXISTS — нужен факт наличия

Вместо JOIN + DISTINCT, когда данные второй таблицы в результате не нужны:

```
SELECT r.response_id
FROM responses AS r
WHERE EXISTS (
    SELECT 1
    FROM calls AS c
    WHERE c.response_id = r.response_id
      AND c.status = 'successful'
);
-- NOT EXISTS — отклики, где такого звонка не было
```

DISTINCT ON — первая строка каждой группы

```
SELECT DISTINCT ON (response_id)
    response_id, event_id, event_at, event_type
FROM events
ORDER BY response_id, event_at, event_type, event_id;
-- поля DISTINCT ON = первые поля ORDER BY, остальные решают, какая строка
останется
```

Аналог `ROW_NUMBER() ... WHERE rn = 1`, но короче. Есть **только в PostgreSQL**.

generate_series — создать строки, которых нет в данных

```
WITH calendar AS (
    SELECT d::date AS day
    FROM generate_series(DATE '2026-08-01', DATE '2026-08-07', INTERVAL '1 day')
AS d
)
SELECT c.day, COUNT(e.event_id) AS events     -- не COUNT(*): иначе пустой день
= 1
FROM calendar AS c
LEFT JOIN events AS e ON e.event_at::date = c.day
GROUP BY c.day
ORDER BY c.day;
```

```
-- свой диапазон для каждой строки: 7 дней после каждого отклика
SELECT r.response_id, d.day::date
FROM responses AS r
CROSS JOIN LATERAL generate_series(
    r.created_at::date, r.created_at::date + 6, INTERVAL '1 day'
) AS d(day);
```

6. Производительность

ГЛАВНЫЙ ПРИНЦИП

Сначала **сократите объём данных**, затем выполняйте тяжёлые операции — JOIN, агрегации и оконные функции.

Чеклист, если запрос стал тяжёлым

Вопрос	Что сделать
Нужны ли все выбранные колонки?	только нужные колонки вместо SELECT *; LIMIT для разведки
Можно ли применить фильтр раньше?	WHERE в подготовительных CTE, до JOIN / UNION / окон
Не размножились ли строки после JOIN?	сверить число строк с гранулярностью
Нет ли лишнего DISTINCT?	найти причину дублей вместо того, чтобы их прятать
Нужны ли все сортировки и окна?	убрать лишние ORDER BY и OVER()
Не считается ли результат несколько раз?	вынести общий расчёт в один CTE

DISTINCT: инструмент, а не лекарство от дублей

Уместен — уникальность и есть вопрос	Костыль
SELECT DISTINCT professional_role — какие роли есть	строк стало больше после JOIN — и непонятно почему
COUNT(DISTINCT employer_id) — сколько разных работодателей	дальше идут SUM или AVG — DISTINCT их не исправит
ROW_NUMBER() / DISTINCT ON — известные дубли, чистим осознанно	SELECT DISTINCT * «на всякий случай»

СОВЕТ

Простой тест: можете объяснить, **откуда дубли** и **почему их правильно схлопнуть**? Да — DISTINCT уместен. Нет — найдите причину и исправьте её до JOIN: агрегируйте источник заранее или используйте EXISTS.

EXPLAIN

`EXPLAIN` показывает план выполнения запроса, `EXPLAIN ANALYZE` — ещё и фактическое время. Пригодится, когда базовые проверки не помогли.

Шпаргалка

Перед запросом: какой бизнес-вопрос? что такое одна строка итога? какие промежуточные шаги? какой инструмент на каждом шаге?

Задача	Конструкция
Промежуточный шаг	`WITH step AS (SELECT ...) SELECT ... FROM step`
Сложить события	`SELECT ... UNION ALL SELECT ...  -- типы совпадают`
Факт наличия	`WHERE EXISTS (SELECT 1 FROM t WHERE ...)`
Показатель рядом со строкой	`AVG(x) OVER (PARTITION BY g)`
Накопительно	`COUNT(*) OVER (PARTITION BY g ORDER BY t ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`
Первая строка группы	`ROW_NUMBER() OVER (PARTITION BY g ORDER BY t, id) = 1  -- через CTE`
То же в PostgreSQL	`SELECT DISTINCT ON (g) ... ORDER BY g, t, id`
Рейтинг	`RANK() / DENSE_RANK() OVER (PARTITION BY g ORDER BY x DESC)`
Группы по n	`NTILE(n) OVER (ORDER BY x)`
Соседняя строка	`LAG(x) / LEAD(x) OVER (PARTITION BY g ORDER BY t, id)`
Интервал в часах	`EXTRACT(EPOCH FROM t2 - t1) / 3600`
Первое / последнее	`FIRST_VALUE(x) / LAST_VALUE(x) ... ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`
Часть строки	`SPLIT_PART(s, '|', n)::integer`
Строки → строка	`STRING_AGG(x, ' → ' ORDER BY t)`
Очистить текст	`REGEXP_REPLACE(s, '[^0-9]', '', 'g')`
Поиск по шаблону	`col ~* 'шаблон'`
Строки → массив	`ARRAY_AGG(x ORDER BY t)`
Размер / вхождение	`CARDINALITY(arr) / 'v' = ANY(arr)`
Массив → строки	`CROSS JOIN UNNEST(arr) WITH ORDINALITY AS u(x, pos)`
Поле из JSONB	`(meta ->> 'key')::type`
Ключ / структура в JSONB	`meta ? 'key' / meta @> '{"k": "v"}'`
Календарь	`generate_series(DATE '...', DATE '...', INTERVAL '1 day')`
Медиана, перцентиль	`PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x)`